Introduction to Deep Learning

Session 2: Perceptron, MLPs & Activations

March 2026

Magda Gregorová - magda.gregorova@thws.de

Technical University
of Applied Sciences
Würzburg-Schweinfurt

Licensed according to CC-BY-SA 4.0

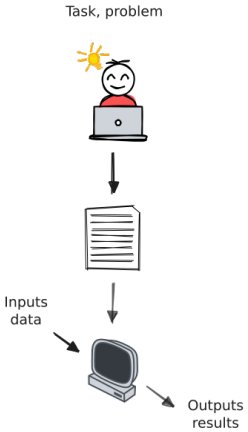
Lecture Overview

1. Supervised Learning

2. Regression - Linear / Nonlinear

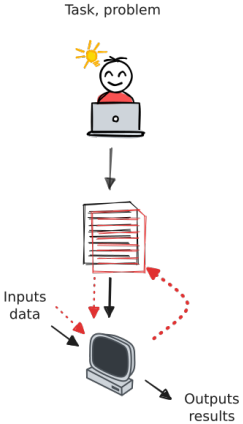
Why Machine Learning

Traditional Software



adapt to changing conditions
too many variations to capture
too complex to understand fully
too expensive to compute

Machine Learning

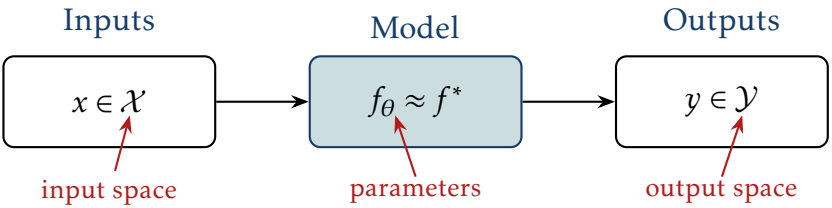


Supervised Learning

Setup: data pairs $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x} \in \mathcal{X}$, $y \in \mathcal{Y}$

Assumption: exists true relationship $f^* : \mathcal{X} \to \mathcal{Y}$, $y = f^*(\mathbf{x})$, but f^* is never observed!

Goal: learn $f_\theta : \mathcal{X} \to \mathcal{Y}$ from $\mathcal{D}$ such that $f_\theta \approx f^*$ $\Rightarrow$ predict on **unseen** inputs $\mathbf{x} \notin \mathcal{D}$

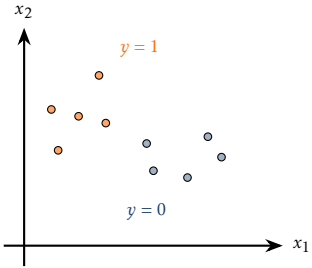


Notation: $f_\theta(\mathbf{x}) = f(\mathbf{x}; \theta) = \hat{y}$

Supervised Learning - Basic Tasks

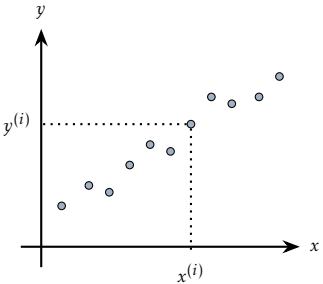
Classification (binary)

- Output: discrete label $\mathcal{Y} = \{0, 1\}$
- Goal: find $f_\theta : \mathcal{X} \to \{0, 1\}$
- Predict category for input



Regression

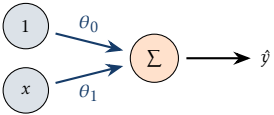
- Output: continuous value $\mathcal{Y} = \mathbb{R}$
- Goal: find $f_\theta : \mathcal{X} \to \mathbb{R}$
- Predict numerical value for input

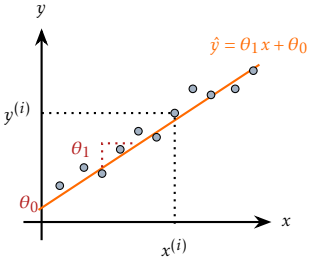


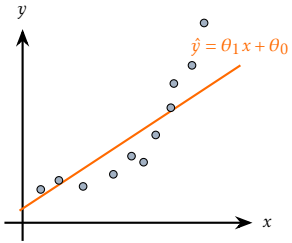
Regression and Linear Model

Simplest model: linear function

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1)$$







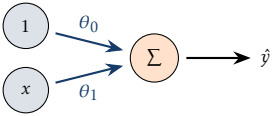
Limitation: real data is rarely linear

Solution: more expressive f_θ

How: $f_\theta = \sin, \log, \sqrt{\ }, \exp, \ldots$ **?**

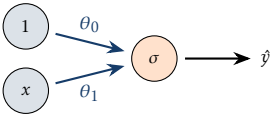
Nonlinear Regression

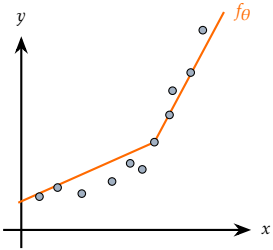
Linear: $\hat{y} = f_\theta(x) = \theta_1 x + \theta_0$

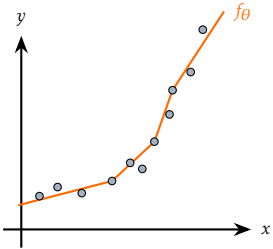


Nonlinear: $\hat{y} = f_\theta(x) = \sigma(\theta_1 x + \theta_0)$

$\sigma : \mathbb{R} \to \mathbb{R}$ - nonlinear activation function







Idea: piecewise linear functions

Nonlinear Regression

Linear: $\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1)$

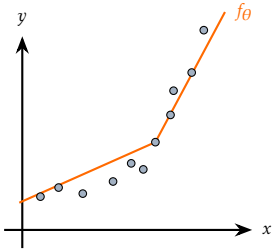
Nonlinear: $\hat{y} = f_\theta(x) = \sigma(\theta_1 x + \theta_0)$

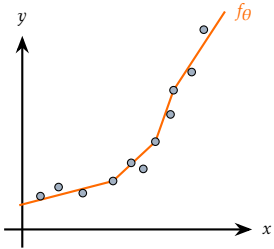
$\sigma : \mathbb{R} \to \mathbb{R}$ - nonlinear activation function

Idea: piecewise linear functions

How: more breaks, more expressive f_θ

Key result: approximate any continuous func

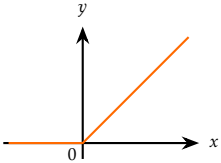




Building block

Rectified Linear Unit

$\mathrm{ReLU}(x) = \max(0, x)$

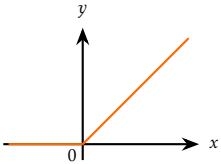


Linear Function + ReLU

Building block

Rectified Linear Unit

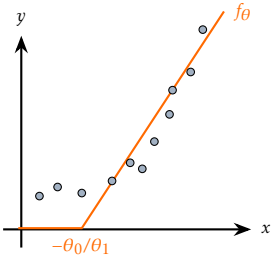
$\mathrm{ReLU}(x) = \max(0, x)$



Effect: one bend at $x = -\theta_0/\theta_1$

Limitation: only one breakpoint - not expressive enough

Solution: combine multiple ReLUs

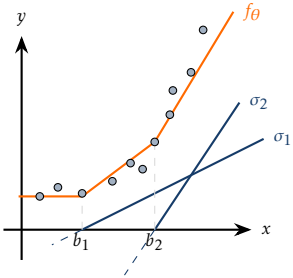


Combining Multiple ReLUs

Combine: weighted sum of ReLU functions

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \theta_{11}^{(2)}\sigma_1 + \theta_{12}^{(2)}\sigma_2$$

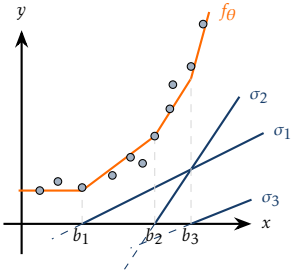
$$\sigma_1 = \mathrm{ReLU}(\theta_{11}^{(1)}x + \theta_{10}^{(1)}) \quad \sigma_2 = \mathrm{ReLU}(\theta_{21}^{(1)}x + \theta_{20}^{(1)})$$



Effect: one bend per ReLU term

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \theta_{11}^{(2)}\sigma_1 + \theta_{12}^{(2)}\sigma_2 + \theta_{13}^{(2)}\sigma_3$$

$$\sigma_3 = \mathrm{ReLU}(\theta_{31}^{(1)}x + \theta_{30}^{(1)})$$



Key result: more breakpoints $\Rightarrow$ more expressive f_θ

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \sum_j \theta_{1j}^{(2)}\,\sigma\left(\theta_{j1}^{(1)}x + \theta_{j0}^{(1)}\right)$$

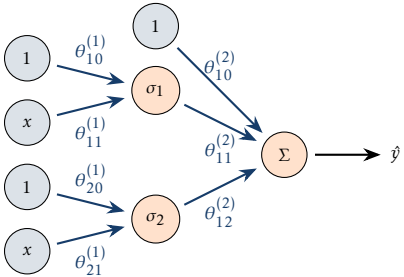
Two Linear Functions + ReLU

Mathematical representation :

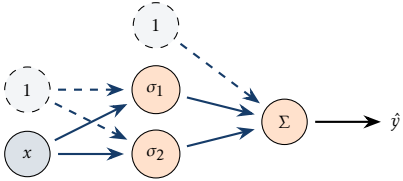
$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \theta_{11}^{(2)}\sigma_1 + \theta_{12}^{(2)}\sigma_2$$

$$\sigma_1 = \mathrm{ReLU}(\theta_{11}^{(1)}x + \theta_{10}^{(1)}) \quad \sigma_2 = \mathrm{ReLU}(\theta_{21}^{(1)}x + \theta_{20}^{(1)})$$

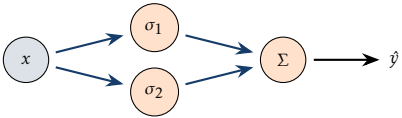
Graphical representation:



Simplified graphical representation:



Most simple graphical representation:



One Layer Neural Network

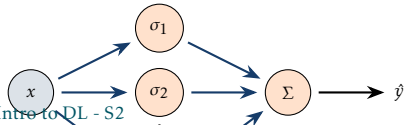
Multiple neurons, single input

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \sum_{j=1}^{k} \theta_{1j}^{(2)} \, \sigma\left(\theta_{j1}^{(1)} x + \theta_{j0}^{(1)}\right)$$

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \sum_{j=1}^{k} \theta_{1j}^{(2)} \, \sigma\left(\theta_{j1}^{(1)} x + \theta_{j0}^{(1)}\right)$$

$$\hat{y} = \boldsymbol{\Theta}^{(2)} \, \sigma\left(\boldsymbol{\Theta}^{(1)} x + \boldsymbol{\theta}^{(1)}\right) + \theta_{10}^{(2)}$$

Layers: input hidden output



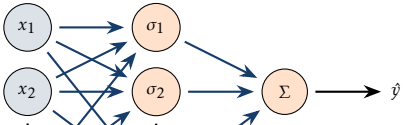
Multiple neurons, multiple inputs

$$\hat{y} = f_\theta(\mathbf{x}) = \theta_{10}^{(2)} + \sum_{j=1}^{k} \theta_{1j}^{(2)} \, \sigma\left(\sum_{i=1}^{d} \theta_{ji}^{(1)} x_i + \theta_{j0}^{(1)}\right)$$

$$\hat{y} = f_\theta(\mathbf{x}) = \theta_{10}^{(2)} + \sum_{j=1}^{k} \theta_{1j}^{(2)} \, \sigma\left(\sum_{i=1}^{d} \theta_{ji}^{(1)} x_i + \theta_{j0}^{(1)}\right)$$

$$\hat{y} = \boldsymbol{\Theta}^{(2)} \, \sigma\left(\boldsymbol{\Theta}^{(1)} \mathbf{x} + \boldsymbol{\theta}^{(1)}\right) + \theta_{10}^{(2)}$$

Layers: input hidden output



From biological to artificial neuron

Biological neuron

- Dendrites receive signals
- Cell body integrates inputs
- Axon fires if threshold exceeded

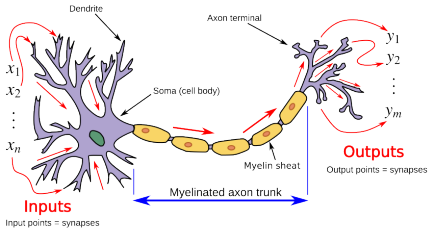
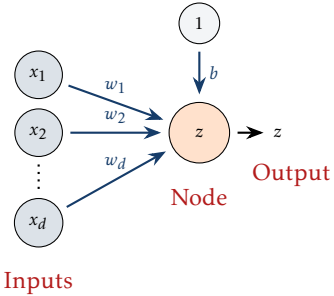


Image: Egm4313.s12 at English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons

Artificial neuron $f : \mathbb{R}^d \to \mathbb{R}$

- Inputs: $\mathbf{x} \in \mathbb{R}^d$
- Params: weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$
- Output: $z = f(\mathbf{x}) = \sum_i w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b$



This is a neural network

[mlp_1hidden.tex]

- Each hidden neuron computes:

$$z_j^{(1)} = w_j x + b_j, \quad a_j^{(1)} = \mathrm{ReLU}(z_j^{(1)})$$

- Output layer combines:

$$f(x;\theta) = \sum_j v_j a_j^{(1)} + c$$

This is a neural network

[mlp_1hidden.tex]

- Each hidden neuron computes:

$$z_j^{(1)} = w_j x + b_j, \quad a_j^{(1)} = \mathrm{ReLU}(z_j^{(1)})$$

- Output layer combines:

$$f(x;\theta) = \sum_j v_j a_j^{(1)} + c$$

- In matrix form:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$
$$\mathbf{a}^{(1)} = \mathrm{ReLU}(\mathbf{z}^{(1)})$$
$$f(\mathbf{x};\theta) = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

- Parameters: $\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}\}$

Universal approximation

Universal Approximation Theorem

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a non-constant, bounded, continuous activation function. Then for any continuous function $g : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exists a 1-hidden-layer neural network $f(\cdot;\theta)$ such that:

$$\sup_{\mathbf{x}\in[0,1]^d} |f(\mathbf{x};\theta) - g(\mathbf{x})| < \varepsilon$$

What it tells us

- MLPs are universal approximators
- Enough neurons can fit any continuous function

What it doesn't tell us

- How many neurons are needed
- How to find the parameters θ
- Whether the model will generalize

Classification

[linear_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

Classification

[xor_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

- Not all problems are linearly separable
  - XOR: four points, no linear boundary works

Classification

[xor_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

- Not all problems are linearly separable
  - XOR: four points, no linear boundary works
- Neural network with ReLU: non-linear decision boundary
- Same architecture as for regression

Activation functions

[activation_functions.tex]

What makes a good activation?

- Non-linear — otherwise depth buys nothing
- Differentiable — needed for backpropagation
- Computationally cheap

Common choices

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}} \in (0,1)$
- **Tanh**: $\tanh(z) \in (-1,1)$
- **ReLU**: $\max(0,z)$ — sparse, fast, but dying neurons

Dying ReLU problem

- Neurons with $z < 0$ output 0 and have zero gradient
- Can become permanently inactive

Notation summary

Symbol	Meaning	Dimensions
$\mathbf{x}$	input	$\mathbb{R}^d$
y	target output	$\mathbb{R}$ or $\{0, 1\}$
θ	all parameters	—
$\mathbf{W}^{(l)}$	weight matrix, layer l	$\mathbb{R}^{d_l \times d_{l-1}}$
$\mathbf{b}^{(l)}$	bias vector, layer l	$\mathbb{R}^{d_l}$
$\mathbf{z}^{(l)}$	pre-activation, layer l	$\mathbb{R}^{d_l}$
$\mathbf{a}^{(l)}$	post-activation, layer l	$\mathbb{R}^{d_l}$
σ	activation function	$\mathbb{R} \to \mathbb{R}$
L	number of hidden layers	—

Forward pass: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \quad \mathbf{a}^{(0)} = \mathbf{x}$